

FIELD MANUAL · CHROMIUM CONTRIBUTION

크롬팀 첫 PR 완전 정복

시니어 엔지니어가 알아야 할
Chromium 기여 프로세스의 모든 것

GitHub가 아니라 Gerrit. PR이 아니라 CL. merge가 아니라 Commit Queue. Chromium 공식 기여 가이드(chromium.googlesource.com)와 depot_tools 소스를 근거로, 환경 세팅부터 CL 업로드, OWNERS 리뷰, trybot 검증, CQ 랜딩, 리버트 대응까지 첫 코드 변경이 main에 들어가는 전 과정을 실전 순서로 정리했다.

근거 자료

[chromium/src docs/contributing.md](#)
[depot_tools git_cl.py](#) 소스

대상 독자

Chromium 신규 기여자
(사내 시니어 전입 가정)

작성일

2026-07-17

목차

- 0. TL;DR — 한 장 요약
- 1. Chromium 세계관 — GitHub가 아니다
 - 1.5 주변 환경 — 커뮤니케이션 채널 지도
- 2. 시작 전 준비 — 계정, CLA, 디자인 독
- 3. 개발 환경 — depot_tools와 체크아웃
- 4. 브랜치와 커밋 — 변경 만들기
- 5. CL 업로드 — git cl upload 해부
- 6. CL 설명문 규격 — 커밋 메시지가 곧 계약
- 7. 리뷰 시스템 — Gerrit 라벨의 정치학
- 8. OWNERS — 코드의 소유권 지도
- 9. 자동 검증 — presubmit과 trybot
- 9.5 로컬 빌드와 테스트 — CL 전의 자기 검증
- 10. Commit Queue — 랜딩의 마지막 관문
- 11. 리버트와 리랜드 — 뒤집혔을 때의 예절
- 12. 스택드 CL — 의존하는 변경 다루기
- 13. 코드 가이드라인 — Chromium의 설계 철학
 - 13.5 Gerrit 웹 UI 실전 지도
- 14. 시니어가 피해야 할 함정 10가지
- 15. 첫 PR 액션 플랜
- 부록 A. git cl 치트시트와 푸터 레퍼런스

0. TL;DR — 한 장 요약

Chromium에서 "PR"은 존재하지 않는다. 존재하는 것은 **CL(Changelist)**이고, 그 CL은 Gerrit에서 리뷰되고, trybot 군단이 검증하고, Commit Queue가 랜딩한다. 전체 흐름은 다음과 같다.

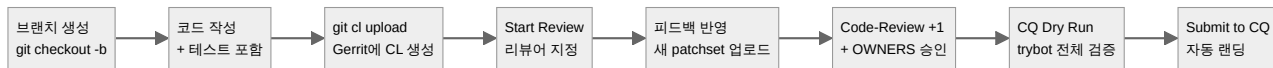


그림 0-1 CL의 생애주기 — 브랜치에서 랜딩까지

표 0-1 GitHub 지식의 Chromium 번역표

GitHub에서 하던 것	Chromium에서는
Pull Request	CL (Changelist) — Gerrit의 변경 단위
fork + push	직접 브랜치, <code>git cl upload</code> 가 Gerrit에 푸시
Approve	Code-Review +1 라벨 (커미터만 부여 가능)
Merge 버튼	Commit-Queue +2 라벨 → CQ가 자동 커밋
CI 체크	trybot (LUCI) — CQ Dry Run으로 트리거
CODEOWNERS	OWNERS 파일 — 디렉터리별 소유권, 승인 강제

가장 중요한 한 가지. 업로드 후 반드시 Gerrit 웹에서 **"Send and Start Review"** 버튼을 눌러야 한다. 누르지 않으면 아무도 리뷰하러 오지 않는다 — 공식 가이드가 경고 표시로 강조할 만큼 가장 흔한 신규자 실수다 (§ 7.2).

1. Chromium 세계관 — GitHub가 아니다

1.1 왜 하필 Gerrit인가

Chromium은 세계 최대급 단일 코드베이스 중 하나다. 수천 명의 기여자, 매일 수백 개의 CL이 동시에 진행되는 규모에서 GitHub의 PR 모델은 감당이 안 된다. Gerrit은 **"한 커밋 = 한 리뷰 단위"**를 강제하고, 커밋을 식별하는 `Change-Id` 로 수정 이력(patchset)을 추적하며, OWNERS 기반 승인을 디렉터리 단위로 강제한다.

1.2 용어 정리

표 1-1 Chromium 고유 용어

용어	의미
CL	Changelist. 리뷰 단위가 되는 하나의 변경. Perforce 시절 유산
Gerrit	리뷰 시스템. chromium-review.googlesource.com
patchset	같은 CL의 수정 버전. 리뷰 반영할 때마다 번호가 올라감

OWNERS	각 디렉터리의 소유자 목록 파일. 승인 권한의 근거
커미터 (committer)	Code-Review +1을 줄 수 있는 권한을 가진 개발자
trybot	LUCI 인프라의 테스트 봇. 여러 플랫폼에서 빌드+테스트 수행
CQ (Commit Queue)	승인된 CL을 검증 후 자동 랜딩하는 시스템
tree (trunk)	main 브랜치. "tree is closed" = 랜딩 중단 상태
depot_tools	Chromium 개발용 Git 래퍼 도구 모음 (gclient, git-cl 등)

1.3 저장소 지도

체크아웃하면 `src/` 아래에 Chromium 본체와 별도 프로젝트들이 공존한다. `v8/`, `third_party/skia/` 같은 종속 프로젝트는 **자체 기여 워크플로**를 갖는다는 점에 주의 — Chromium 가이드가 그대로 적용되지 않는다. ChromiumOS 역시 Chromium을 서브 저장소로 포함하는 별도 세계다.

1.4 시니어로서의 마인드셋

시니어 전입자가 흔히 범하는 착각은 "코드 실력만 있으면 된다"는 것이다. Chromium에서 커밋은 기술적 행위인 동시에 **사회적 행위**다: OWNERS에게 리뷰를 구하고, 디자인 독으로 합의를 모으고, CQ를 통과시키는 전 과정이 협업 기술을 요구한다. 공식 가이드의 첫 조언이 "Communicate"(코드 전에 먼저 대화하라)인 이유다.

1.5 주변 환경 — 커뮤니케이션 채널 지도

Chromium에서 "누구에게 물어보는가"는 코드만큼 중요하다. 채널마다 성격이 다름으로 용도에 맞게 골라 쓴다.

표 1-2 커뮤니케이션 채널과 적합한 용도

채널	용도
버그 트래커 (issues.chromium.org)	긴 논의, 이력 추적, 스크린샷 첨부, 관련 버그 연결. CL 설명문의 Bug:로 연결되는 앵커
Gerrit 코멘트	코드 그 자체에 대한 논의. 설계 논쟁이 길어지면 버그나 디자인 독으로 옮기는 게 예의
디자인 독 메일링 리스트 (chromium-design-docs@)	공개 디자인 독의 제안과 피드백
영역별 논의 그룹	새 기능 아이디어의 사전 검토 (contributing.md 의 Communicate 원칙)
OWNERS 파일 + git log	"이 코드 누가 아는가"의 1차 정보원. 사람 찾기는 여기서 시작
watchlists	관심 파일/영역의 변경을 이메일로 구독. 담당 영역의 맥박을 잡는 도구

1.5.2 사내 전입자의 추가 채널

공개 문서는 여기까지지만, 사내에는 `go/` 링크로 접근하는 남부 자원이 있다 — `contributing.md`도 CLA 검증(`go/cla`), 사내 디자인 독 프로세스(`go/chrome-dd-review-process`) 같은 남부 링크를 참조한다. 입사 첫 주에 팀 온보딩 문서에서 이 링크들을 확보해 두는 것이 좋다.

1.5.3 비동기 예절

글로벌 프로젝트이므로 모든 소통은 기본적으로 비동기다. 가이드는 "리뷰 지연 최소화 가이드라인"을 따로 둘 만큼 시차를 고려한 문화를 강조한다: 질문은 한 번에 모아서, 컨텍스트는 링크로, 상대방이 깨어있을 때를 기다리는 인내심을 갖는다. re-ping은 영업일 1일 이후의 정당한 권리다.

2. 시작 전 준비 — 계정, CLA, 디자인 독

2.1 계정과 법적 절차

표 2-1 사전 준비 체크리스트

항목	내용
Gerrit 계정	chromium-review.googlesource.com 에 Google 계정으로 로그인, git 설정의 이메일과 Gerrit 선호 이메일 일치 필수
CLA	Contributor License Agreement. 개인은 온라인 서명, 법인은 별도 양식. Googler는 사내 CLA로 커버됨
AUTHORS 파일	외부 기여자는 첫 패치에 AUTHORS 항목 추가를 포함해야 함 (Googler는 *@google.com 와일드카드로 면제)
git 설정	user.name, user.email, core.autocrlf false, gerrit.host true

참고: 리뷰어는 외부 기여자의 CL을 LGTM하기 전에 AUTHORS 등재 여부와 CLA 유효성을 확인해야 하는 의무가 있다. 사내 전입자는 해당 없지만, 나중에 당신이 리뷰어가 되면 이 체크리스트를 수행하게 된다.

2.2 코드 전에 대화하라 — Communicate

공식 가이드의 첫 섹션은 코드가 아니라 소통이다. 원칙은 세 가지:

- **새 기능 아이디어**라면? 코드를 쓰기 전에 해당 논의 그룹에 제안한다.
- **기존 코드베이스 변경**이라면? 해당 코드의 OWNERS 파일에 있는 사람들과 먼저 논의한다.
- **긴 논의·스크린샷·이력**이 필요하다면? 버그 트래커(issues.chromium.org)에 버그를 연다. CL 설명문은 버그 링크로 연결한다. 단, 버그가 있다고 패치가 받아들여지는 건 아니다.

2.3 디자인 독이 필요한 경우

다음 중 하나에 해당하면 코드 전에 디자인 독(템플릿 존재)이 요구된다:

- Chromium 전체에 큰 영향을 미치는 변경 (페이지 로딩, 렌더링 같은 크리티컬 패스)
- 역사적 기록 가치가 있는 대규모 작업 — 기준은 대략 **1인-월(1 person-month) 이상**

공개 디자인 독은 chromium-design-docs@chromium.org 로 본다. 시니어 전입자라면 첫 과제가 이 기준에 해당할 확률이 높다 — "디자인 독 없이 3주 코딩"은 Chromium에서 가장 비싼 실수다.

시니어의 첫 주 체크리스트. ① Gerrit 로그인·이메일 정렬 ② git 설정 완료 ③ 담당 영역의 OWNERS 파일 읽기 ④ 팀의 진행 중 디자인 독 목록 파악 ⑤ 첫 과제가 디자인 독 필요 조건인지 TL과 확인.

3. 개발 환경 — depot_tools와 체크아웃

3.1 depot_tools — Chromium의 도구 상자

Chromium 개발은 표준 Git만으로는 불가능하다. **depot_tools**는 Git 위에 얹힌 도구 모음으로, 코드 체크아웃부터 리뷰 업로드까지 전 과정을 담당한다. 주요 구성원:

표 3-1 depot_tools 핵심 도구

도구	역할
fetch	코드베이스 첫 체크아웃. <code>fetch chromium</code> 한 줄로 전체 소스 확보
gclient	멀티 저장소 동기화. <code>gclient sync</code> 로 <code>third_party</code> 갱신 + 훅 실행
git cl	CL 생애주기 관리. <code>upload/try/issue/owners/presubmit</code> 등 서브커맨드
git new-branch	CL용 새 브랜치 생성 (upstream 추적 포함)
git rebase-update	로컬 브랜치들을 upstream에 맞춰 일괄 리베이스
gn / autoninja	빌드 설정 생성 / 컴파일 실행

3.2 체크아웃과 최신 상태 유지

```
# 최초 한 번 — 빈 디렉터리에서
fetch chromium          # src/ 트리 전체 + depot_tools 설정

# 일상적인 동기화 루틴
cd src
git rebase-update       # 내 로컬 브랜치들을 최신 main에 맞춤
gclient sync            # third_party 종속 저장소 갱신 + 빌드 훅 실행
```

`gclient sync` 를 게을리하면 `third_party`와 본체가 어긋나 "나만 안 되는 빌드"가 된다. 하루 시작 루틴에 넣어라.

3.3 작업 공간의 위생

Gerrit 모델에서는 **브랜치 하나 = CL 하나**다. 한 브랜치에 커밋을 쌓더라도 `git cl upload` 는 기본적으로 스쿼시해 하나의 CL로 올린다. 따라서 작업마다 브랜치를 파는 습관이 필수이며, `git cl issue` 로 현재 브랜치가 어떤 CL과 연결됐는지 항상 확인할 수 있다.

4. 브랜치와 커밋 — 변경 만들기

4.1 브랜치 생성

```
git checkout -b mychange -t origin/main
# 또는 depot_tools 스타일
git new-branch mychange
```

4.2 코드 작성 시 지킬 것

- **스타일 가이드 준수.** Chromium은 C++ 스타일 가이드가 별도 문서로 존재하며, presubmit이 기계적 위반을 걸러낸다.
- **테스트 동반.** "모든 변경은 대응하는 테스트를 포함해야 한다"가 공식 규칙이다. 테스트 없는 CL은 리뷰에서 가장 먼저 지적된다.
- **리뷰 가능한 크기.** 가이드의 경고: "리뷰 시간은 패치 크기에 따라 지수적으로 증가한다." 큰 변경은 스택드 CL로 쪼갬다 (§ 12).

4.3 로컬 커밋

```
git add -A
git commit
```

커밋 메시지는 나중에 `git cl upload` 가 CL 설명문으로 가져가고, 랜딩 시 그대로 영구 커밋 메시지가 된다. 처음부터 규격 (§ 6)에 맞춰 쓰는 게 수고를 덜다.

5. CL 업로드 — git cl upload 해부

5.1 업로드가 실제로 하는 일

```
git cl upload
```

이 한 줄 뒤에서 일어나는 일 (depot_tools `git_cl.py` 소스 기준):

1. **인증 확인** — Gerrit 자격 증명과 ReAuth 요건을 검사. 느린 presubmit을 다 돌린 뒤 인증 실패로 죽는 참사를 막기 위해 먼저 확인한다.
2. **설명문 작성** — 에디터가 열리고 CL 설명문을 입력받는다. 브랜치명이 `bug-123`, `fix-123-foo` 형태면 버그 번호를 자동 추출해 `Bug:` 푸터에 채운다.
3. **watchlist CC 추가** — 변경한 파일 경로를 구독 중인 사람들을 자동으로 CC에 넣는다.
4. **presubmit 실행** — 스타일·저작권 헤더·금지 패턴 등 로컬 검사. 실패하면 업로드 중단.
5. **Gerrit 푸시** — `refs/for/main` 으로 푸시해 CL 생성. `Change-Id` 가 커밋 메시지에 자동 삽입된다.

5.2 유용한 플래그

```
git cl upload -r foo@chromium.org,bar@chromium.org # 리뷰어 지정
git cl upload -b 123456 # Bug: 푸터
git cl upload -x 123456 # Fixed: 푸터 (랜딩
시 버그 자동 종결)
git cl upload -d # CQ dry run 즉시 트
리거
git cl upload -c # 승인 즉시 CQ 투입
git cl upload -a # Auto-Submit (승인
후 자동 CQ)
git cl upload --r-owners # 관련 OWNERS 전원을
리뷰어로
git cl upload --dependencies # 의존 브랜치 CL들까지
함께 재업로드
```

5.3 업로드 성공 후

```
remote: New Changes:
remote:   https://chromium-review.googlesource.com/c/chromium/src/+/1485
699 ...
```

이 URL이 CL의 집이다. `git cl issue`로 언제든지 다시 확인할 수 있다. 단, 업로드만으로는 아무도 알림을 받지 않는다 — § 7의 "Start Review"가 필요하다.

5.4 흔한 장애: HTTP 401

`git cl upload`가 HTTP 401로 죽는 알려진 이슈가 있다. 핵의는 `git config --global http.postBuffer 524288000` 설정, 또는 시간을 두고 최신 동기화 후 재시도. 공식 문서에 실려 있을 만큼 빈번하니 당황하지 말 것.

6. CL 설명문 규격 — 커밋 메시지가 곧 계약

6.1 기본 형식

Summary of change (one line)

Longer description of change addressing as appropriate: why the change is made, context if it is part of many changes, description of previous behavior and newly introduced differences, etc.

Long lines should be wrapped to 72 columns for easier log message viewing in terminals.

Bug: 123456

규칙의 핵심 셋: ① **한 줄 요약 + 빈 줄** — `git log --oneline` 등 도구가 첫 줄을 제목으로 쓰기 때문. ② 본문은 72열 줄바꿈. ③ 푸터(footer)는 반드시 **메시지의 마지막 단락**에 연속으로 — 빈 줄이 끼면 무시된다.

6.2 수동 테스트 지침: Test: 푸터

Test: Load `example.com/page.html` and click the foo-button; see `crbug.com/123456` for more details.

trybot이 커버하지 못하는 수동 검증이 있다면 테스터가 따라 할 수 있게 명시한다. 자동화 테스트가 전부라면 생략 가능.

6.3 주요 푸터 레퍼런스

표 6-1 CL 푸터 요약 (전체 목록은 부록 A)

푸터	의미
Bug:	관련 버그. 콤마로 여러 개, Bug: skia:1234 처럼 프로젝트 지정 가능
Fixed:	Bug와 같되, 랜딩 시 버그를 자동으로 fixed 상태로 종결
Test:	수동 테스트 수행 내역
Change-Id:	업로드 시 자동 삽입. 같은 CL의 patchset을 묶는 고유 ID
Cq-Include-Trybots:	기본 세트 외 추가로 돌릴 trybot 지정 (bucket:builder 형식)
No-Presubmit: true	presubmit 생략 (주로 자동 리버트용)
Reviewed-by:	랜딩 시 Gerrit이 자동 추가 — 승인자 기록
Cr-Commit-Position:	랜딩 시 자동 추가 — SVN식 단조 증가 리비전 번호

이전 CL 참조는 `https://crrev.com/c/NUMBER` 형식으로 쓰는 것이 관례. R= 푸터는 폐기(deprecated)되었으니 `git cl upload -r` 플래그를 쓸 것.

7. 리뷰 시스템 — Gerrit 라벨의 정치학

7.1 리뷰어는 어떻게 고르는가

원칙: "리뷰어는 커미터여야 하고, 해당 코드를 잘 아는 커미터여야 한다." 모르겠으면 변경 파일에서 가장 가까운 상위 OWNERS 파일의 사람에게 묻는다.

- `git cl owners` — OWNERS 파일 기반으로 리뷰어를 자동 추천
- Gerrit UI의 "Suggest Owners" 버튼 — Reply 다이얼로그에서 제공, 특별한 이유 없으면 이 추천을 따르는 게 가이드의 공식 권장
- 파일마다 리뷰어는 기본 1명 — "누가 먼저든 해주겠지"라고 여러 명을 걸면 ① 중복 리뷰로 낭비되거나 ② 서로 미루며 아무도 안 하는 두 실패 모드가 있다는 게 가이드의 경고

7.2 리뷰 시작 — 가장 흔한 신규자 실수

⚠ 공식 가이드의 대문자 경고. "Be sure to click the 'Send and Start Review' button as **NO ONE WILL LOOK AT YOUR CHANGE UNTIL YOU DO SO**". 업로드 직후 CL은 WIP(작업 중) 상태일 수 있으며, 리뷰어를 지정하고 "Send and Start Review"를 눌러야 비로소 알림이 간다.

리뷰 요청 후 응답은 **영업일 1일 이내**에 오는 게 기대치다. 없으면 re-ping하라 — 이것도 가이드에 명문화된 권리다. 피드백 반영은 로컬 브랜치에 커밋을 쌓고 `git cl upload`를 다시 치면 patchset이 올라간다.

7.3 승인의 2층 구조

표 7-1 CL 제출에 필요한 두 종류의 승인

승인 종류	누가 주는가	필요 수량
Code-Review +1	아무 커미터 (작성자 본인 제외)	커미터 작성자: 1명 / 비커미터: 2명
Code-Owners 승인	변경된 각 파일의 OWNER	모든 파일 각각 — OWNER가 +1하거나 작성자가 그 파일의 OWNER

즉 "아무 커미터의 +1"과 "디렉터리 소유자의 승인"은 별개다. CL이 디렉터리 세 개를 건드리면 세 곳의 OWNERS 커버리지가 필요하다. 또한 **모든 리뷰 코멘트가 resolved**여야 제출 가능하다 — +1과 함께 달린 잔여 코멘트도 응답하거나 ACK 버튼으로 해소해야 한다.

7.4 새 patchset을 올리면 승인은 어떻게 되나

표 7-2 patchset 간 승인 이월(sticky) 규칙

상황	Code-Review +1
단순 리베이스 (충돌 없음)	유지됨
커밋 메시지만 변경	유지됨
커미터 작성자, 변경 파일 목록 동일	유지됨
비커미터의 코드 변경	소실 — 승인 재획득 필요

7.5 리베이스 예절

리뷰 도중 최신화가 필요할 때의 매너: ① 이해결 코멘트 반영분을 **먼저** patchset으로 올리고, ② 리베이스만 한 것을 **별도 patchset**으로 올린다. 리뷰어가 "코멘트 반영 diff"와 "리베이스 diff"를 분리해 볼 수 있게 하는 배려다. 그리고 Chromium은 글로벌 프로젝트다 — 리뷰어와 시차가 12시간인 게 기본값임을 잊지 말 것.

8. OWNERS — 코드의 소유권 지도

8.1 OWNERS 파일의 구조

각 디렉터리에는 `OWNERS` 파일이 있고, 그 디렉터리(및 하위) 코드의 승인 권한자를 나열한다. GitHub CODEOWNERS와 유사하지만 훨씬 오래됐고 강제력이 강하다: **모든 파일은 OWNER의 +1 또는 작성자 자신의 OWNER 자격이 필요하다.**

8.2 소유권의 실전적 의미

- **승인 강제** — OWNERS enforcement는 mandatory다. "대충 아는 사람 +1"로는 랜딩이 불가능하다.
- **리뷰 라우팅** — OWNERS는 승인자인 동시에 "이 코드를 누구에게 물어봐야 하는가"의 전화부다. 파일의 git log와 함께 본다.
- **일부 OWNERS 파일의 그룹 규칙** — `foo-reviews` 같은 리뷰 로테이션 그룹이 등재된 경우가 있고, 그 쪽으로 보내는 게 선호된다. 인라인 주석에 안내가 있지만 Gerrit UI에는 표시되지 않으니 파일을 직접 열어볼 것.

8.3 커미터십 — +1 권한의 계층

Chromium 세계에서 권한은 단계적이다: 일반 기여자(CL 작성 가능) → try job 접근(trybot 실행 가능) → 커미터(Code-Review +1 부여 가능). 시니어 전임자도 Chromium 커미터십은 별도 취득 과정이며, 보통 몇 개의 CL을 성공적으로 랜딩한 뒤 리뷰어의 추천으로 지명된다. 커미터가 아닌 동안은 내 CL에 +1을 줄 커미터 두 명이 필요하다는 점을 기억하라.

9. 자동 검증 — presubmit과 trybot

9.1 presubmit — 로컬의 첫 관문

`git cl upload` 시점에 presubmit 스크립트가 돌아 기계적 오류(스타일 위반, 저작권 헤더 누락, 금지 API 사용 등)를 걸러낸다. 수동으로도 실행 가능하다:

```
git cl presubmit
```

```
# 현재 CL에 대해 로컬 검사 실행
```

9.2 trybot — LUCI의 테스트 군단

본격 검증은 trybot이다. 각 trybot은 특정 플랫폼(Linux/Windows/Mac/Android, ASAN 빌드 등)에서 패치를 적용해 Chromium을 컴파일하고 테스트를 돌린다. 트리거는 Gerrit 우측 상단의 **CQ Dry Run** 버튼 — Commit-Queue +1 라벨과 동일하다.

표 9-1 trybot 상태 버블의 색상 언어

색	의미
회색	아직 시작 안 함
노랑	실행 중 — 나중에 다시 확인
볼록	봇 자체의 예외 (인프라 문제). 내 패치 탓이 아닌 경우가 대부분 — Dry Run 재시도
빨강	테스트 실패 — 봇을 클릭해 원인 분석
초록	통과

try job 실행 권한이 없다면: @chromium.org 계정은 셀프 신청, 외부 기여자는 리뷰어에게 대행 요청. 시니어 전입자라면 사내 계정으로 신청하면 된다.

9.3 추가 trybot이 필요할 때

기본 세트에 없는 특수 빌드(예: 특정 안드로이드 ASAN)가 필요하면 `Cq-Include-Trybots: luci.chromium.try:android-asan` 푸터로 지정한다. Gerrit의 Checks 탭 "Choose Tryjobs" UI에서 고르고 푸터 문법을 복사할 수 있다.

9.5 로컬 빌드와 테스트 — CL 전의 자기 검증

9.5.1 빌드 체인: gn과 autoninja

Chromium은 수만 개의 타깃으로 구성되므로 전체 빌드는 거의 하지 않는다. 변경 영역의 타깃만 빌드하는 것이 일상이다:

```
gn gen out/Default          # 빌드 디렉터리 생성 (최초 1회)
autoninja -C out/Default chrome      # 브라우저 전체
autoninja -C out/Default unit_tests  # 특정 테스트 바이너리만
```

out/Default 는 관례적 이름일 뿐, out/Debug , out/Release 등 설정별 디렉터리를 병행할 수 있다. gn 인자 (args.gn)로 컴포넌트 빌드, 심볼 레벨 등을 조정하며, `is_component_build = true` 는 링크 시간을 크게 줄여 개발 루프를 빠르게 한다.

9.5.2 테스트의 계층

표 9-2 Chromium 테스트 계층 (대표 예시)

계층	예시	특징
----	----	----

단위 테스트	<code>unit_tests</code> , <code>base_unittests</code>	가장 빠르고 국소적. CL의 기본 방어막
브라우저 테스트	<code>browser_tests</code> , <code>content_browsertests</code>	실제 브라우저 인스턴스 기동. UI/통합 동작 검증
웹 플랫폼 테스트	<code>web_tests</code> (구 <code>layout tests</code>)	렌더링/웹 표준 적합성. WPT와 연동

```
out/Default/unit_tests --gtest_filter="MySuite.MyTest" # 특정 테스트만 실행
```

리뷰어는 "컴파일되고 테스트를 통과하는 코드"를 기대한다(§ 7.2). 로컬에서 관련 테스트를 돌리지 않은 CL은 trybot의 시간을 낭비하고 리뷰어의 신뢰를 소모한다.

9.5.3 필터로 반복을 빠르게

대형 테스트 바이너리도 gtest 필터로 수초 만에 원하는 것만 돌릴 수 있다. 수정 → 빌드 → 필터 테스트의 루프를 짧게 유지하는 것이 Chromium 개발의 생산성 핵심이고, 전체 테스트는 trybot/CQ에 위임한다.

10. Commit Queue — 랜딩의 마지막 관문

10.1 CQ의 역할

모든 승인(CR+1 + OWNERS + 코멘트 해소)이 끝나면, Gerrit 우측 상단 **Submit to CQ** (= Commit-Queue +2 라벨)로 CQ에 투입한다. CQ는 다시 한번 전체 trybot을 돌리고, 전부 초록이면 **자동으로 main에 커밋**한다. "Merge 버튼을 누르는 사람"이 없는 세계다.

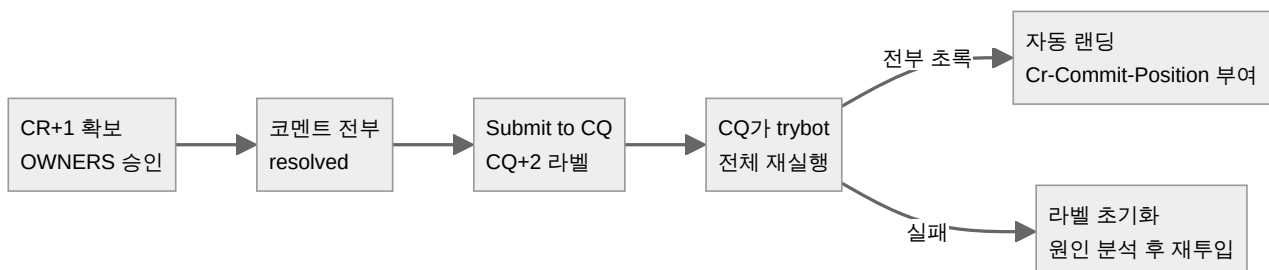


그림 10-1 Commit Queue 처리 흐름

10.2 실패 대응

- **머지 충돌** — 최신 main과 충돌하면 CQ가 거부한다. 리베이스 후 재업로드.
- **flaky 테스트** — 내 변경과 무관한 간헐 실패는 CQ 재투입으로 통과되는 경우가 많다. 단, 반복되면 진짜 원인일 수 있으니 무시 금지.
- **tree closed** — main이 불안정해 랜딩이 동결된 상태. 기다리는 수밖에 없다. (자동 리버트용 `No-Tree-Checks: true` 같은 우회 푸터는 일반 개발자용이 아니다.)

10.3 비상 탈출구와 그 자격

"비상시에는 커밋 접근 권한자가 CQ와 모든 안전망을 우회해 직접 커밋할 수 있다"고 가이드는 인정한다 — 이른바 chumping. 단, 이것은 재난 대응용이며 남용은 신뢰를 태운다. 시니어라도 첫 해에는 CQ를 정상 경유하는 것만으로 충분하다.

11. 리버트와 리랜드 — 뒤집혔을 때의 예절

11.1 리버트는 실패가 아니다

CQ를 통과하고 랜딩된 변경도 나중에 리버트되는 일이 흔하다. 원인은 다양하다 — 예상 못한 다른 변경과의 충돌, CQ가 커버하지 않는 테스트의 회귀, 플래키한 타이밍 이슈. 가이드의 원문: *"don't be discouraged! This can be a common part of the Chromium development cycle."*

11.2 리랜드(reland) 절차

1. **Create Reland 버튼** — 원본 CL에서 클릭. 원본과 동일한 diff의 새 CL이 만들어지고, 커밋 메시지에 원본으로의 추적 링크가 남는다. 이 "종이 흔적" 덕에 나중에 gardener(트리 감시자)가 회귀 디버깅을 할 수 있다.
2. 수정은 **후속 patchset으로** — 리랜드에 수정이 필요하면 첫 patchset(=원본과 동일) 이후의 patchset으로 올린다. 리뷰어가 첫 patchset과 최신을 비교해 "리랜드 수정분만" 볼 수 있다.
3. **커밋 메시지에 수정 요약** — "무엇이 달라져서 이번엔 안전한가"를 명시: "included needed fix", "disabled failing tests", "crash was fixed elsewhere". 수정이 별도 선행 CL로 처리됐다면 그 CL을 링크한다.

시니어의 리버트 문화. 리버트는 인신공격이 아니라 트리 건강을 위한 표준 절차다. 반대로 당신이 다른 사람의 CL을 리버트해야 할 때도 같은 원칙 — 빠르게 뒤집고, 원인 분석은 랜딩된 트리를 지킨 뒤에.

12. 스택드 CL — 의존하는 변경 다루기

12.1 왜 스택인가

큰 기능을 CL 하나로 올리면 리뷰가 지수적으로 느려진다 (§ 4.2). 핵의는 작은 CL의 연쇄다. Gerrit의 **relation chain**은 부모 CL 대비 diff를 보여주므로, 리뷰어는 각 단계를 독립적으로 승인할 수 있다.

12.2 내 부모 CL 위에 쌓기

```
git checkout featureA
git checkout -b featureB
git branch --set-upstream-to featureA # 핵심: upstream을 부모 브랜치로
# ... 코드 작성, 커밋 ...
git cl upload # B의 CL이 A의 CL에 스택됨
```

12.3 남의 CL 위에 쌓기

```
git cl patch --force <parent-CL-number> # 부모 CL을 로컬로 가져옴
git checkout -b featureB
git branch --set-upstream-to change-<parent-CL-number>
# ... 코드 작성, 커밋 ...
git cl upload
```

주의: `change-<번호>` 브랜치에서 직접 업로드하면 원저자의 CL에 새 patchset이 올라가버린다. 반드시 새 브랜치를 파서 작업한다. 부모 CL에 새 patchset이 올라오면:

```
git checkout change-<parent-CL-number>
git fetch origin <최신 patchset sha> && git reset --hard FETCH_HEAD
```

12.4 스택 운영 규칙

- 스택 전체를 재업로드할 때: `git cl upload --dependencies`가 의존 브랜치들을 순서대로 돌며 재업로드한다.
- 부모가 랜딩되면 자식을 main으로 리베이스 — `git rebase-update`가 이 반복 작업을 자동화한다.
- 스택이 꼬였다(diverged)는 업로드 에러가 나면 `git rebase-update [-n]`으로 정리.

13. 코드 가이드라인 — Chromium의 설계 철학

contributing.md의 "Code guidelines" 섹션은 절차가 아니라 세계관이다. 시니어가 리뷰에서 "왜 이렇게까지?"라는 질문을 받기 전에 알아야 할 원칙들:

표 13-1 Chromium 코드 가이드라인 요약

원칙	실전적 의미
코드는 Chromium 내 다른 코드를 위해 존재해야 한다	"언젠가 쓰일" 유틸리티는 거절된다. 모든 코드에 실제 소비자가 있어야 함
중앙 위치(//base 등) 이동은 다수 소비자가 있을 때만	과도한 공통 라이브러리는 코드 비대와 복잡성을 낳는다

기존 코드는 지금 하려는 일을 위해 설계되지 않았다	새 기능을 억지로 끼우지 말고 리팩터링으로 자리를 만들어라
공통 코드는 모두의 책임	여러 서브시스템이 만나는 파일은 가장 취약하다 — 복잡성 추가에 각별히 주의
모든 변경은 테스트 동반	자동화 테스트가 프로젝트의 전진 방식 그 자체. "Protect your code with tests"
지원 언어 세트를 유지	새 언어 도입은 //ATL_OWNERS 승인 필요. 유지보수성이 무엇보다 우선
새 OS/아키텍처/툴체인/최상위 디렉터리는 chrome-atls@ 승인	장기 유지보수 비용 때문. 기존 ifdef 분기를 최대한 재사용
API 변경/마이그레이션은 요구 팀이 80% 이상 수행	외부효과 감소 원칙 — 변경을 원하는 팀이 비용의 대부분을 부담
AI 코딩 도구 사용 시 Chromium AI 정책 준수	별도 정책 문서 존재 — 시대상의 반영

"80% 규칙"의 교훈. 이 규칙은 scikit-learn 분석에서 본 "변경하는 사람이 하위를 고친다"는 구글의 Live-at-Head 철학과 같은 뿌리다. 대규모 코드베이스의 생존 비결은 결국 **변경 비용을 변경을 일으킨 쪽이 지게 하는 가** 격표라는 것.

13.5 Gerrit 웹 UI 실전 지도

CLI로 다 되는 것처럼 보여도, 리뷰의 상당 부분은 Gerrit 웹에서 일어난다. 시니어가 첫날부터 익숙해져야 할 화면 요소들:

13.5.1 CL 페이지의 핵심 요소

표 13-2 Gerrit CL 화면의 요소와 기능

요소	기능
Start Review / Reply	리뷰 개시와 코멘트 발송. 리뷰어·CC 추가도 여기서
Suggest Owners	Reply 다이얼로그 내 링크. OWNERS 기반 리뷰어 자동 추천
CQ Dry Run / Submit to CQ	우측 상단. trybot 트리거와 CQ 투입 버튼
Patchset 선택기	두 patchset 간 diff 비교. "PS1 vs latest"로 리뷰 반영분만 보기
Checks 탭	trybot 결과 상세. Choose Tryjobs로 추가 봇 선택 가능
라벨 영역	Code-Review, Commit-Queue 등 현재 투표 상태
Relation chain	스택드 CL의 부모·자식 관계 표시

13.5.2 리뷰어로서의 화면 사용법

- 코멘트는 **draft** 상태로 쌓인다 — 파일별로 달은 코멘트는 Reply→Send 전까지 공개되지 않는다. 중간에 저장땀려 볼 수 있어 신중한 리뷰가 가능하다.
- +1과 코멘트의 분리** — "+1, 그리고 사소한 코멘트 몇 개"는 흔한 조합. 작성자는 ACK 버튼으로 확인했음을 표시해 코멘트를 해소할 수 있다 (§ 7.3).

- **Unresolved만 보기** — 코멘트 스레드가 길어지면 미해결 스레드만 필터링하는 것이 실무 표준.

13.5.3 대시보드와 검색

Gerrit 대시보드는 "Outgoing Reviews"(내가 올린 CL)와 "Incoming Reviews"(내가 리뷰할 CL)로 나뉜다. 검색 연산자(`owner:me status:open`, `reviewer:me`)를 쓰면 자신만의 작업 큐를 만들 수 있다. 시니어는 보통 커스텀 대시보드로 팀의 CL을 감시한다.

14. 시니어가 피해야 할 함정 10가지

1. 업로드 후 Start Review를 안 누른다 — 알림이 안 가서 CL이 며칠간 방치된다. 가장 흔한 실수 (§ 7.2).
2. 디자인 독 없이 큰 작업을 시작한다 — 1인-월 이상이면 디자인 독 필수. 코드 3주 치가 리뷰에서 뒤집힌다 (§ 2.3).
3. CL을 비대하게 만든다 — 리뷰 시간은 크기의 지수함수. 스택드 CL로 쪼개라 (§ 12).
4. 파일마다 리뷰어를 여러 명 건다 — 중복 리뷰 또는 상호 회피의 함정. 파일당 1명이 원칙 (§ 7.1).
5. 테스트 없이 CL을 올린다 — "모든 변경은 테스트 동반"이 공식 규칙. 리뷰어의 첫 지적 사항 (§ 4.2).
6. 승인 후 코드를 고치고 재승인을 안 받는다 — 비커미터는 코드 변경 시 승인 소실. sticky 규칙을 오해하지 말 것 (§ 7.4).
7. gclient sync를 게을리한다 — third_party 불일치로 "나만 안 되는 빌드"가 된다 (§ 3.2).
8. 푸터를 메시지 중간에 쓴다 — 마지막 단락 외의 푸터는 무시된다. Bug:가 인식 안 되는 이유 대부분이 이것 (§ 6.1).
9. flaky 실패를 무한 재시도한다 — 두 번 이상 같은 실패면 재시도가 아니라 분석이다 (§ 10.2).
10. 리버트를 인신공격으로 받아들인다 — Chromium의 일상이다. 리랜드 절차를 배워라 (§ 11).

15. 첫 PR 액션 플랜

15.1 주차별 로드맵

표 15-1 첫 CL까지의 4주 계획

주차	목표
1주차	환경 구축: depot_tools, fetch chromium, 첫 빌드 성공, Gerrit 로그인·이메일 정렬, git 설정. 담당 영역 OWNERS 파일과 팀 디자인 독 읽기
2주차	관찰: 팀원 CL 3~5개를 끝까지 따라가며 리뷰 톤과 라벨 흐름 학습. 작은 버그 하나 골라 TL과 사전 논의 (Communicate 원칙)
3주차	첫 CL: 작고 테스트 포함된 변경. 업로드 → Start Review → OWNERS 추천 리뷰어 지정 → 피드백 반영
4주차	랜딩: 승인 획득 → CQ Dry Run 초록 → Submit to CQ → 자동 랜딩 확인. 이후 try job 접근 신청, 두 번째 CL은 스택드로 도전

15.2 첫 CL 당일 체크리스트

1. `git rebase-update && gclient sync` — 최신 상태에서 시작
2. `git checkout -b fix-123456-desc -t origin/main` — 버그 번호 포함 브랜치명
3. 코드 + 테스트 작성, 로컬 빌드·테스트 통과
4. 커밋 메시지 규격 확인: 한 줄 요약, 빈 줄, 72열, 마지막 단락에 `Bug: 123456`
5. `git cl presubmit` 로컬 통과
6. `git cl upload` → 출력된 Gerrit URL 확인
7. 웹에서 Suggest Owners로 리뷰어 지정 → **Send and Start Review** 클릭
8. CQ Dry Run으로 trybot 확인 → 초록 확인
9. 승인 후 Submit to CQ → 랜딩 확인 → 팀에 공유

15.3 랜딩 이후 — 다음 단계

첫 랜딩은 시작일 뿐이다. 이후의 성장 경로는 대략 이렇다:

- **try job 접근 취득** — 스스로 trybot을 돌릴 수 있게 되면 리뷰어 의존이 줄어든다.
- **리뷰어로 참여** — 커미터가 아니어도 코멘트는 달 수 있다. 팀 CL을 읽고 질문하는 것이 영역 지식을 쌓는 지름길이다.
- **커미터 지명** — 꾸준한 양질의 CL이 쌓이면 리뷰어가 지명한다. 이제 당신도 +1을 줄 수 있다.
- **OWNERS 등재** — 특정 디렉터리의 깊은 기여가 인정되면 OWNERS 파일에 이름이 오른다. 그 순간부터 그 코드의 승인 책임도 당신의 것이다.

15.4 스스로 묻는 다섯 질문

이 CL은 충분히 작은가? 테스트가 있는가? 설명문이 "왜"를 답하는가? 리뷰어가 누구인지 OWNERS가 말해주는가? 리버트돼도 설명할 수 있는가?

다섯 질문에 모두 답할 수 있으면, 그 CL은 이미 Chromium의 기준을 충족한다.

16. 부록 A. git cl 치트시트와 푸터 레퍼런스

A.1 git cl 서브커맨드 치트시트

명령	용도
<code>git cl upload [-r 리뷰어] [-b 버그]</code>	CL 생성/새 patchset 업로드
<code>git cl issue [번호]</code>	현재 브랜치의 CL URL 확인/연결 변경
<code>git cl owners</code>	OWNERS 기반 리뷰어 추천
<code>git cl presubmit</code>	로컬 presubmit 검사
<code>git cl try</code>	trybot 트리거
<code>git cl patch <CL번호></code>	남의 CL을 로컬에 적용

<code>git cl split -d</code> 설명파일	브랜치를 OWNERS 단위로 쪼개 여러 CL로 업로드
<code>git cl archive</code>	종결된 CL의 로컬 브랜치 정리
<code>git new-branch <이름></code>	작업 브랜치 생성
<code>git rebase-update</code>	모든 로컬 브랜치를 upstream에 맞춰 갱신

A.2 푸터 완전 레퍼런스

푸터	의미 / 형식
Bug:	관련 버그, 콤마 구분, 프로젝트:번호 가능. 사내 트래커는 b: 접두
Fixed:	Bug와 동일 + 랜딩 시 자동 종결
Test:	수동 테스트 내역 (자유 형식)
Change-Id:	자동 삽입. CL 식별자
Cq-Include-Trybots:	추가 trybot, bucket:builder 형식
No-Presubmit: true	presubmit 생략 (자동 리버트용)
No-Try: true	tryjob 생략 (자동 리버트용)
No-Tree-Checks: true	단한 트리에도 제출 (자동 리버트용)
Reviewed-by:	랜딩 시 자동 — 승인자 기록
Reviewed-on:	랜딩 시 자동 — 리뷰 페이지 링크
Cr-Commit-Position:	랜딩 시 자동 — ref@{#번호} 형식의 리비전 번호
Cr-Branched-From:	비-main 브랜치 랜딩 시 자동 — 분기점 기록

A.3 자주 묻는 질문 (FAQ)

질문	답
브랜치에 커밋이 여러 개인데 CL은 어떻게 되나?	기본적으로 하나의 CL로 스쿼시된다. 커밋 단위 리뷰가 필요하면 브랜치를 나눠 스택드 CL로 (§ 12)
리뷰어가 3일째 무응답이다	영업일 1일 이후 re-ping은 정당한 권리. 그래도 없으면 OWNERS의 다른 사람이나 TL에게 라우팅 변경 상담
+1을 두 명 받았는데 제출이 안 된다	Code-Review +1과 OWNERS 승인은 별개다. 모든 파일의 OWNERS 커버리지와 코멘트 해소를 확인 (§ 7.3)
CQ가 계속 flaky하게 실패한다	한두 번은 재투입. 반복되면 실패 로그에서 내 변경과의 연관성을 분석. 무관하면 해당 테스트의 플래키 버그를 파일링
커밋 메시지만 고치고 싶다	메시지만의 변경은 sticky — CR+1이 유지된다. 코드를 건드리면 규칙이 달라지니 주의 (§ 7.4)
third_party 코드를 고치고 싶다	V8, Skia 등 종속 프로젝트는 자체 워크플로가 있다. 그쪽 가이드를 먼저 확인 (§ 1.3)

A.4 근거 자료

- [chromium/src/docs/contributing.md](#) — 본 문서의 주 근거. 계정·업로드·리뷰·CQ·리랜드·가이드라인·푸터 레퍼런스
- [chromium/tools/depot_tools git_cl.py](#) — 업로드 플래그, 버그 번호 자동 추출, 스택 재업로드 동작의 소스 근거
- [Chromium Chronicle #22 \(developer.chrome.com\)](#) — depot_tools 개요, fetch/git cl/gclient sync 루틴
- [ChromiumOS Contributing Guide \(chromium.org\)](#) — Start Review 절차, Suggest Owners 권장, 커미터십 계층

문서 끝. Chromium 프로세스는 진화 중이며, 최신 규칙은 항상 [chromium.googlesource.com](#)의 공식 문서를 확인할 것. 특히 사내 전용 절차([go/](#) 링크들)는 입사 후 남부 문서로 보완해야 한다.